Name:- Shoeb Sheikh USN-CS23O74

Aim: To implement the random forest algorithm and evaluate the model performance using appropriate metrices

1.List item

```
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
```

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
path= "/content/drive/MyDrive/ML_Dataset/banking.csv"
df=pd.read_csv(path)
df.head(5)
```

	age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	previous
0	44	blue-collar	married	basic.4y	unknown	yes	no	cellular	aug	thu	...	1	999	0
1	53	technician	married	unknown	no	no	no	cellular	nov	fri	...	1	999	0
2	28	management	single	university.degree	no	yes	no	cellular	jun	thu	...	3	6	2
3	39	services	married	high.school	no	no	no	cellular	apr	fri	...	2	999	0
4	55	retired	married	basic.4y	no	yes	no	cellular	aug	fri	...	1	3	1

5 rows × 21 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41188 entries, 0 to 41187
Data columns (total 21 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             41188 non-null  int64
 1   job             41188 non-null  object
 2   marital         41188 non-null  object
 3   education       41188 non-null  object
 4   default         41188 non-null  object
 5   housing         41188 non-null  object
 6   loan            41188 non-null  object
 7   contact         41188 non-null  object
 8   month           41188 non-null  object
 9   day_of_week     41188 non-null  object
 10  duration        41188 non-null  int64
 11  campaign        41188 non-null  int64
 12  pdays           41188 non-null  int64
 13  previous        41188 non-null  int64
 14  poutcome        41188 non-null  object
 15  emp_var_rate    41188 non-null  float64
 16  cons_price_idx  41188 non-null  float64
 17  cons_conf_idx   41188 non-null  float64
 18  euribor3m       41188 non-null  float64
 19  nr_employed     41188 non-null  float64
 20  y               41188 non-null  int64
dtypes: float64(5), int64(6), object(10)
memory usage: 6.6+ MB
```

```
df.shape
```

```
(41188, 21)
```

```
df.describe()
```

	age	duration	campaign	pdays	previous	emp_var_rate	cons_price_idx	cons_conf_idx	euribor3
count	41188.00000	41188.000000	41188.000000	41188.000000	41188.000000	41188.000000	41188.000000	41188.000000	41188.00000
mean	40.02406	258.285010	2.567593	962.475454	0.172963	0.081886	93.575664	-40.502600	3.62129
std	10.42125	259.279249	2.770014	186.910907	0.494901	1.570960	0.578840	4.628198	1.73444
min	17.00000	0.000000	1.000000	0.000000	0.000000	-3.400000	92.201000	-50.800000	0.63400
25%	32.00000	102.000000	1.000000	999.000000	0.000000	-1.800000	93.075000	-42.700000	1.34400
50%	38.00000	180.000000	2.000000	999.000000	0.000000	1.100000	93.749000	-41.800000	4.85700
75%	47.00000	319.000000	3.000000	999.000000	0.000000	1.400000	93.994000	-36.400000	4.96100
max	98.00000	4918.000000	56.000000	999.000000	7.000000	1.400000	94.767000	-26.900000	5.04500

```
df.columns
```

```
Index(['age', 'job', 'marital', 'education', 'default', 'housing', 'loan',
       'contact', 'month', 'day_of_week', 'duration', 'campaign', 'pdays',
       'previous', 'poutcome', 'emp_var_rate', 'cons_price_idx',
       'cons_conf_idx', 'euribor3m', 'nr_employed', 'y'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.False_
```

```
df[df.isnull().any(axis = 1)]
```

age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	previous	poutcome	emp

0 rows × 21 columns

```
df.isnull().sum()
```

Show hidden output

```
df[df.isnull().any(axis = 1)]
```

age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	previous	poutcome	emp

0 rows × 21 columns

```
df.info()
```

Show hidden output

```
X = df.drop(columns=['y'])
y = df['y']
```

```
X.head()
```

Show hidden output

```
#One-hot encode categorical variables
# X = pd.get_dumies(X)
X=pd.get_dummies(X, dtype = int)
```

```
X.head()
```

Show hidden output

```
X.shape
```

```
(41188, 63)
```

```
X.columns
```

Show hidden output

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.7, test_size = 0.3, random_state = 42)
```

```
X_train.shape, X_test.shape
```

```
((28831, 63), (12357, 63))
```

```
from sklearn.ensemble import RandomForestClassifier
rf_classifier = RandomForestClassifier(n_estimators =100, n_jobs=-1, max_depth=5, oob_score=True,random_state=42)
rf_classifier.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(max_depth=5, n_jobs=-1, oob_score=True, random_state=42)
```

```
y_pred = rf_classifier.predict(X_test)
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
cm = confusion_matrix(y_test, y_pred)
```

```
print('Confusion Matrix\n')
cm = confusion_matrix(y_pred, y_test)
print(confusion_matrix(y_pred,y_test))
```

```
Confusion Matrix

[[10848  1158]
 [   83   268]]
```

```
print('Accuracy Score\n')
print(accuracy_score(y_pred, y_test))
```

```
Accuracy Score

0.8995710933074371
```

```
rf_classifier.oob_score_
```

```
0.9024660955221809
```

```
print('Classification Report\n')
print(classification_report(y_pred, y_test))
```

```
Classification Report

              precision    recall  f1-score   support

           0       0.99      0.90      0.95     12006
           1       0.19      0.76      0.30       351

    accuracy                           0.90     12357
   macro avg       0.59      0.83      0.62     12357
weighted avg       0.97      0.90      0.93     12357
```

HYPERPARAMETER THEORY

```
from joblib import parallel
## GridSearchCV##

rf = RandomForestClassifier(random_state=42, n_jobs= -1)

params = {
    'max_depth': [2,3,5,10,20],
    'min_samples_leaf': [5,10,20,50,100,200],
    'n_estimators': [10,25,30,50,100,200]
}
```

```
from sklearn.model_selection import GridSearchCV
grid_search = GridSearchCV(estimator=rf, param_grid=params, cv=4, n_jobs=-1, verbose=1, scoring='accuracy')
grid_search.fit(X_train, y_train)
```

```
Fitting 4 folds for each of 180 candidates, totalling 720 fits
```

GridSearchCV

best_estimator_:
RandomForestClassifier

RandomForestClassifier

```
grid_search.best_score_
```

```
np.float64(0.9139467076496568)
```

```
print(grid_search.best_params_)
```

```
{'max_depth': 20, 'min_samples_leaf': 5, 'n_estimators': 200}
```

Start coding or generate with AI.